

TERRALYNX

Autonomous Geospatial Incident Command System
& Disaster Operations Decision Support Platform

INITIAL VERSION

Technical Architecture & Operational Documentation

PREDICTIVE • DETERMINISTIC • GEOSPATIAL • OPERATIONAL

01 / Overview

TERRALYNX is a disaster-management decision support platform designed to move emergency operations from reactive post-crisis response toward predictive, deterministic geospatial planning.

The initial version focuses on extreme hydrometeorological disasters including super cyclones, storm surges, and delta flash floods, with an initial geographic focus on vulnerable coastal and delta regions of Odisha, India.

Primary geographic focus

Mahanadi River Basin • Cuttack Millennium City • CDA sectors • Bhubaneswar

Core concept

LIVE DATA → RISK CALCULATION → EVACUATION & LOGISTICS PLANNING → COMMAND-CENTER DECISION SUPPORT

02 / Technology Stack

BACKEND	Python 3.11/3.13 • FastAPI • Pydantic v2 • Uvicorn • Requests
FRONTEND	React 18 • TypeScript • Vite • Tailwind CSS • Lucide React
MAPPING	MapLibre GL • OpenStreetMap Overpass QL
WEATHER	Open-Meteo live/hourly meteorological telemetry
SEARCH	Photon / Komoot geocoding
DEMOGRAPHICS	Official Government Census demographic profiles
VISUALIZATION	Canvas/GPU wind particles • radar • choropleth risk • inspection HUDs

03 / System Architecture

TERRALYNX separates external data collection, deterministic computation, API delivery, and interface rendering. This prevents live-service changes or failures from silently altering the decision logic.

SYSTEM FLOW

EXTERNAL DATA → SERVICE LAYER → DECISION ENGINES → FASTAPI → REACT / MAPLIBRE

Service layer — obtains and normalizes weather, geospatial, search, demographic, and scenario data.

Decision layer — calculates zone risk, evacuation demand, shelter allocation, routing, and emergency resource requirements.

Presentation layer — renders the operational picture through dashboards, maps, tables, simulations, and grounded AI.

04 / Project Structure

Backend

```
backend/app/main.py – FastAPI endpoint
backend/app/api/router.py – REST API route handlers
backend/app/engine/hazard_impact.py – hazard and risk calculations
backend/app/engine/exposure.py – evacuation demand calculations
backend/app/engine/shelter_optimizer.py – shelter allocation and capacity constraints
backend/app/engine/routing.py – evacuation routes and cutoff calculations
backend/app/engine/resource_planner.py – emergency fleet and resource sizing
backend/app/services/weather_service.py – Open-Meteo integration
backend/app/services/search_service.py – geocoding/search
backend/app/services/dynamic_district_generator.py – OSM-derived sectors
backend/app/services/census_service.py – demographic integration
backend/app/services/scenario_service.py – scenario orchestration
```

Frontend

```
command_center/ – incident command dashboard
map/ – map, weather, demographics, search and wind visualization
resources/ – emergency resource planner
simulator/ – what-if scenarios
shelter/ – shelter optimization
routing/ – evacuation route viewer
ai_assistant/ – grounded decision assistant
```

05 / Decision Models

The mathematical models form the decision-making core of TERRALYNX. Each calculation is expressed below in operational terms so the logic can be understood without needing advanced mathematical notation.

5.1 Zone Risk Score

The system combines five risk factors into a single score from **0 to 100**.

```
RISK SCORE =  
  28% × Rainfall Risk  
+ 22% × Wind Risk  
+ 20% × Storm-Surge Risk  
+ 18% × Low-Elevation Risk  
+ 12% × Vulnerable-Housing Risk
```

Rainfall Risk — 24-hour rainfall compared against a 3.0 reference threshold, capped at 100.

Wind Risk — maximum wind gust compared against a 2.0 reference threshold, capped at 100.

Storm-Surge Risk — surge height × 25.

Low-Elevation Risk — $100 - (\text{elevation} \times 10)$, never below 0.

Vulnerable-Housing Risk — percentage of kutcha housing.

Precise model:

```

$$R(z) = 0.28 \cdot \min(100, P24h/3.0) + 0.22 \cdot \min(100, Wgusts/2.0) + 0.20 \cdot \min(100, Ssurge \cdot 25) + 0.18 \cdot \max(0, 100 - \text{Eelev} \cdot 10) + 0.12 \cdot Kkutcha$$

```

5.2 Assisted Evacuation Demand

The model estimates how many people may need assisted evacuation by combining population, overall risk, and vulnerability.

```
ASSISTED EVACUATION DEMAND =  
  Population × Risk Exposure Multiplier × Vulnerability Factor  
  
VULNERABILITY FACTOR =  
  40% × Vulnerable-Housing Ratio  
+ 35% × Elderly Population Ratio  
+ 25% × Flood Threat Index
```

This gives higher assistance requirements to areas where evacuation is more difficult because of housing vulnerability, elderly populations, or flood exposure.

5.3 Shelter Selection

Shelters are ranked by combining travel distance, safety, and elevation.

```
SHELTER COST =  
  Travel Distance + Safety Penalty - Elevation Benefit
```

A lower cost means a more suitable shelter. Higher safety and elevation improve suitability, while greater travel distance reduces it.

Precise model:

```

$$\text{Cost}(z,s) = \text{HaversineDistance}(z,s) + 0.10 \cdot (100 - \text{SafetyScore}(s)) - 0.20 \cdot \max(0, \text{Elevation}(s) - 5.0m)$$

```

Capacity rule: a shelter can never receive more evacuees than its available capacity. When utilization exceeds 80%, the system can queue high-capacity Temporary Emergency Complexes.

06 / Emergency Resource Planning

TERRALYNX translates evacuation demand into practical resource requirements for transport, rescue, medical response, tactical teams, and sustenance.

6.1 Evacuation Buses

The initial model assumes that 80% of assisted evacuees may require bus transportation.

```
REQUIRED BUSES = People Requiring Bus Evacuation ÷ Effective Bus Capacity
People requiring buses = Total evacuation demand × 80%
Effective capacity = 40 passengers × 1.5 convoy-turnaround factor
Always round upward to a whole bus.
Required buses = ceil((Total evacuation demand × 0.80) / (40 × 1.5))
```

6.2 Rescue Boats & OBMs

For low-elevation delta areas:

```
RESCUE BOATS / OBMs = 4 × Delta-Risk Zones + 2 × Critical Zones
```

The calculation applies to sectors with elevation of 3.5 m or less.

6.3 Medical & Tactical Resources

Advance Life Support ambulances are sized against ICU-transfer requirements and registered medical dependencies. NDRF/ODRAF resources are represented as 10-person tactical units scaled to critical structural-breach zones.

6.4 72-Hour Sustenance

```
72-HOUR RATION REQUIREMENT = Total Evacuation Demand × 3 Daily Packs
```

07 / Core Platform Modules

01 Incident Command Center

Threat banner, CAP emergency-cell broadcast, priority directives, tactical authorization and infrastructure monitoring.

02 Geospatial Risk & Weather Map

Dynamic OSM sectors, live weather inspection, forecast timelines, wind direction, demographics, risk layers and wind particles.

03 Shelter Optimization

Capacity utilization, current/incoming allocations, safety ratings and temporary emergency-complex activation.

04 Evacuation Routing & Cutoff

Road elevation, hydraulic/surge cutoff assessment and elevated arterial bypass routing.

05 Fleet Logistics & Resources

Mobility, rescue, medical and ration requirements, deficit tracking and mutual-aid requisition.

06 What-If Simulator

Precipitation, wind, surge and fleet controls, severe-event presets, differential analysis and automatic mitigation.

07 **Grounded AI Assistant**

Natural-language operational intelligence grounded in the current deterministic simulation state.

08 / API Surface

<code>/api/scenario/current</code>	Get current operational scenario
<code>/api/scenario/simulate</code>	Run a what-if scenario
<code>/api/location/search</code>	Search locations and landmarks
<code>/api/telemetry/point</code>	Get point-specific weather telemetry
<code>/api/ai/query</code>	Query the grounded decision assistant
<code>/api/census/official</code>	Retrieve official demographic information

09 / Data Flow

1. A user selects or searches for a location.
2. OSM and geocoding services resolve the geographic context.
3. Open-Meteo supplies weather and forecast telemetry.
4. Demographic and exposure inputs are associated with the zone.
5. Deterministic engines calculate risk and operational requirements.
6. FastAPI exposes the scenario state.
7. React and MapLibre render the operational picture.
8. The AI assistant explains the resulting state.

10 / Data Integrity & Reliability

External APIs are data providers, not decision engines. Incoming values should be validated, normalized to consistent units, and handled explicitly when unavailable.

A failed weather or geospatial service must not silently produce an unsafe or fabricated decision output. Dynamic OSM geometry should be preferred over arbitrary dummy boundaries in production paths.

Demographic information should retain its source context and should not be represented as live telemetry when it is not.

11 / Grounded AI

The AI assistant is an interpretation layer over the deterministic simulation. It does not replace the mathematical decision engines.

USER QUESTION → CURRENT SCENARIO STATE → RELEVANT METRICS → GROUNDED RESPONSE

The assistant should distinguish calculated values from observations and assumptions, and should not invent weather values, shelter capacity, evacuation orders, or government instructions.

12 / Engineering Principles

- Keep decision calculations deterministic and independently testable.
- Enforce shelter capacity as a hard constraint.
- Separate live telemetry from calculated decision state.
- Prefer dynamic OSM and Open-Meteo data over dummy production data.
- Maintain strict TypeScript types across frontend/backend contracts.
- Use the #0a0d14 dark command-center visual foundation.
- Prevent external-service failures from corrupting decisions.
- Make scenario outputs reproducible from their inputs.

13 / System Concept

TERRALYNX connects changing environmental conditions with explainable operational decisions through a single geospatial command interface.

LIVE DATA → THREAT → RISK → EVACUATION → SHELTERS → ROUTES → RESOURCES → COMMAND DECISION

The initial implementation establishes the foundation for a production-grade geospatial incident-command platform while preserving the deterministic integrity of its decision engines.

TERRALYNX — Initial Version